

Guaranteed SLAs

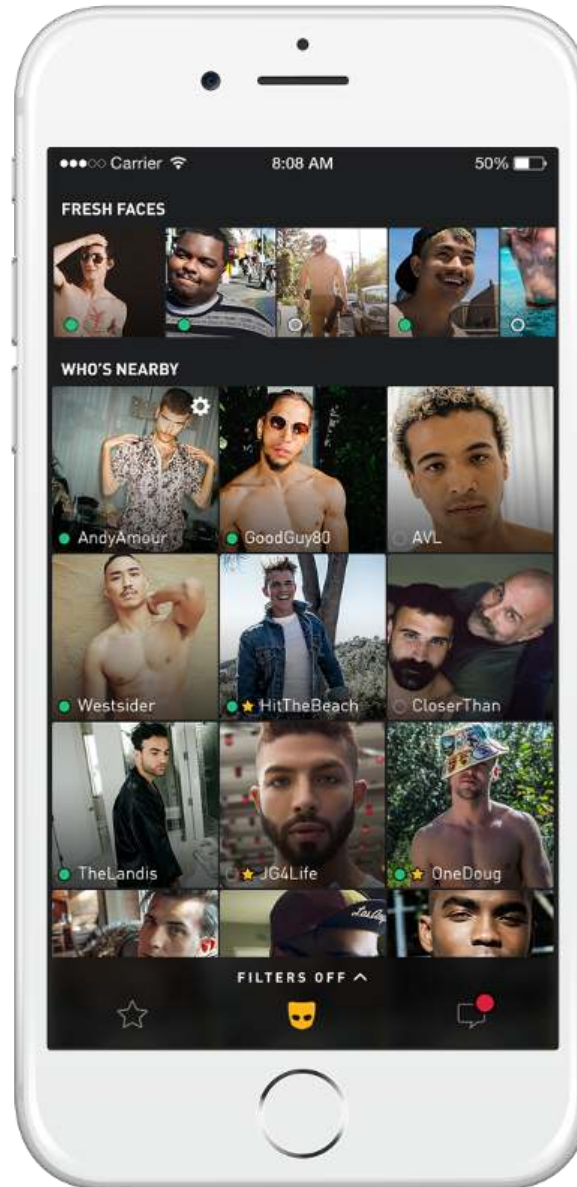


with higher-order functions, and no Magic Numbers!

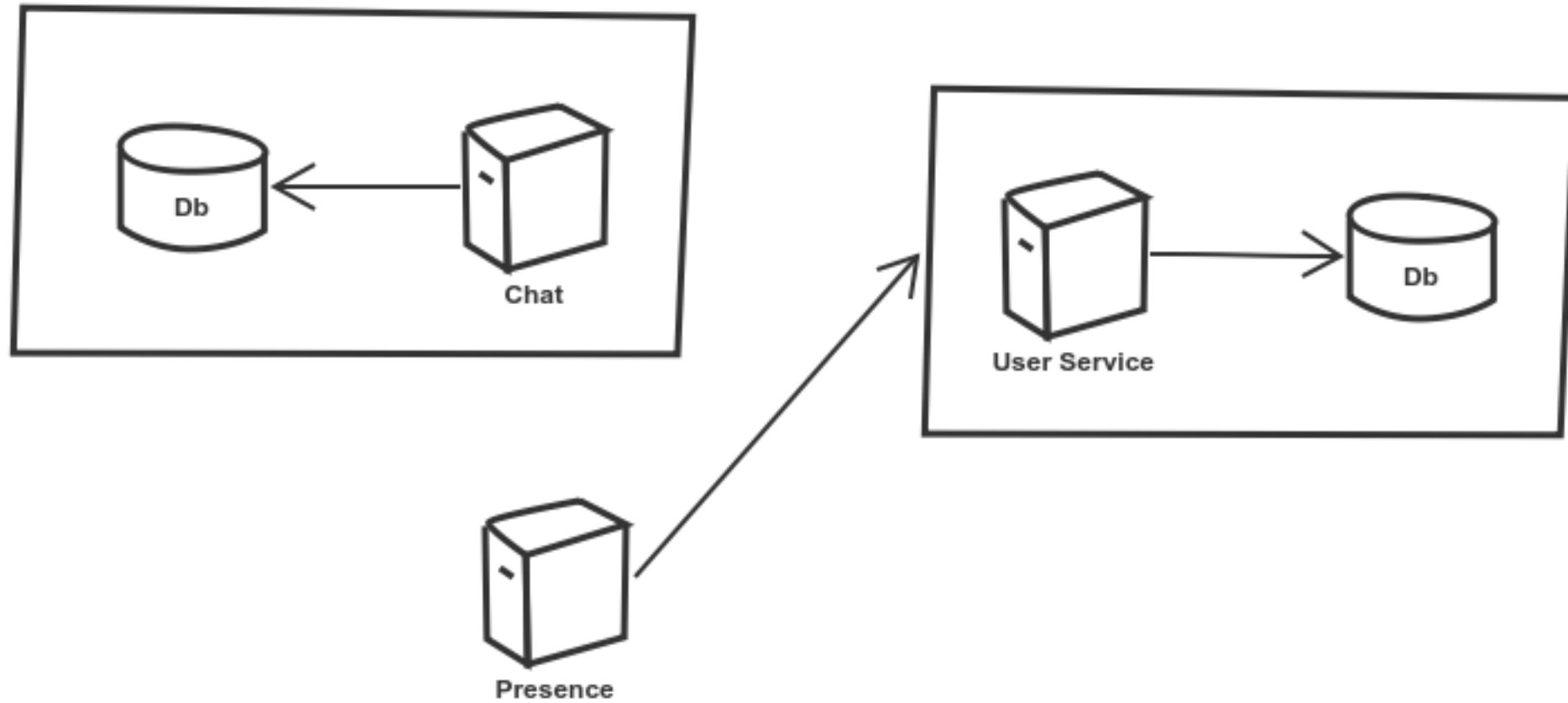
Rafał Studnicki
Lambda Days 2018

Introduction

What is Grindr?



Backend stack





Utilization Law

$$\rho = \lambda / \mu$$

utilization rate

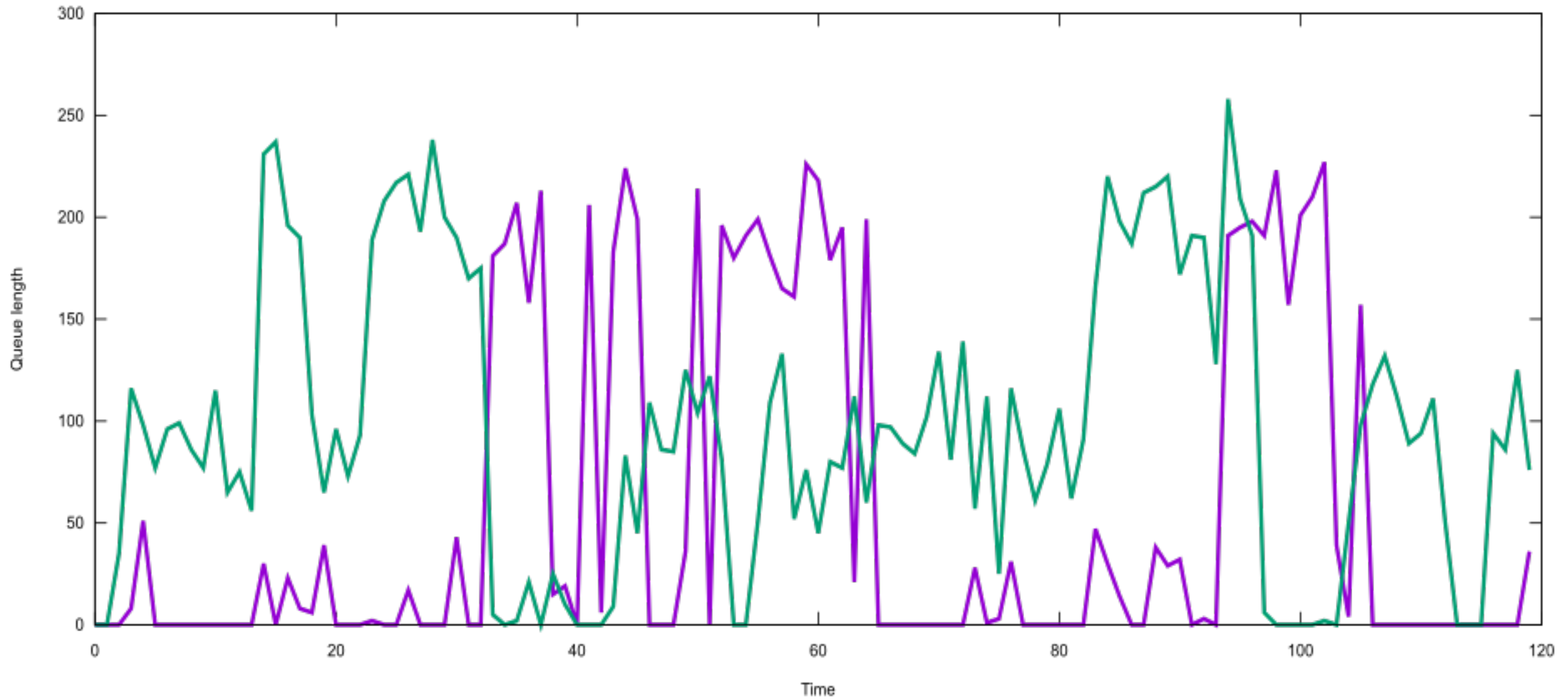
incoming rate of requests

average service rate

The diagram shows the Utilization Law equation $\rho = \lambda / \mu$. Three orange arrows point from text labels below to the variables in the equation: one from 'utilization rate' to ρ , one from 'incoming rate of requests' to λ , and one from 'average service rate' to μ .

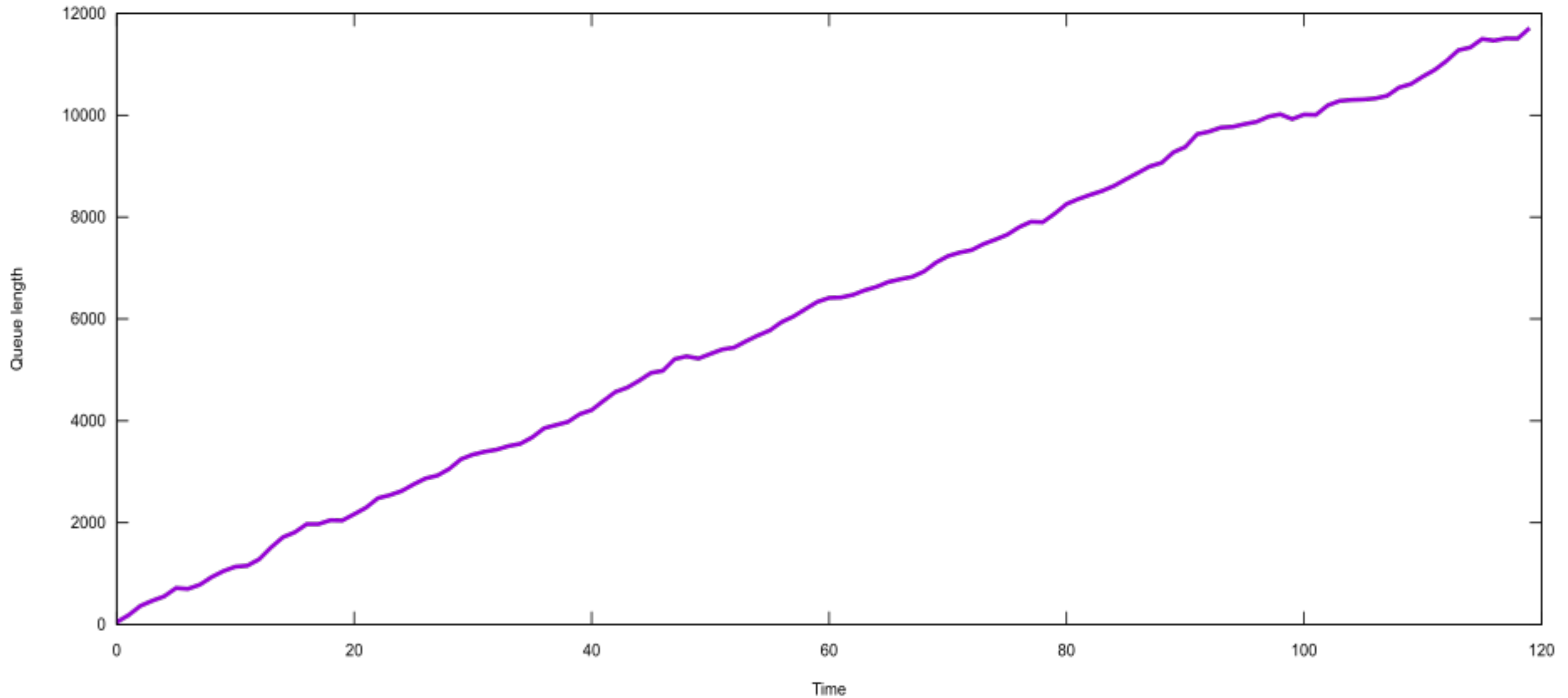


Utilization Law illustrated $\lambda = 1000$ (purple $\mu = 2000$, green $\mu = 1100$)

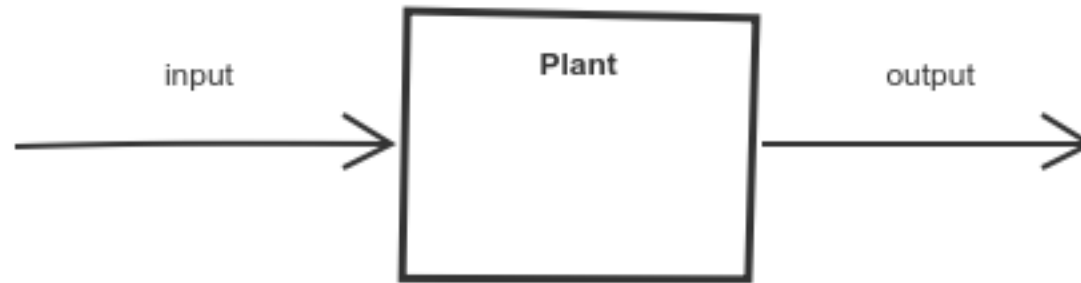




Utilization Law illustrated ($\lambda=1000$, $\mu=900$)



Taking control

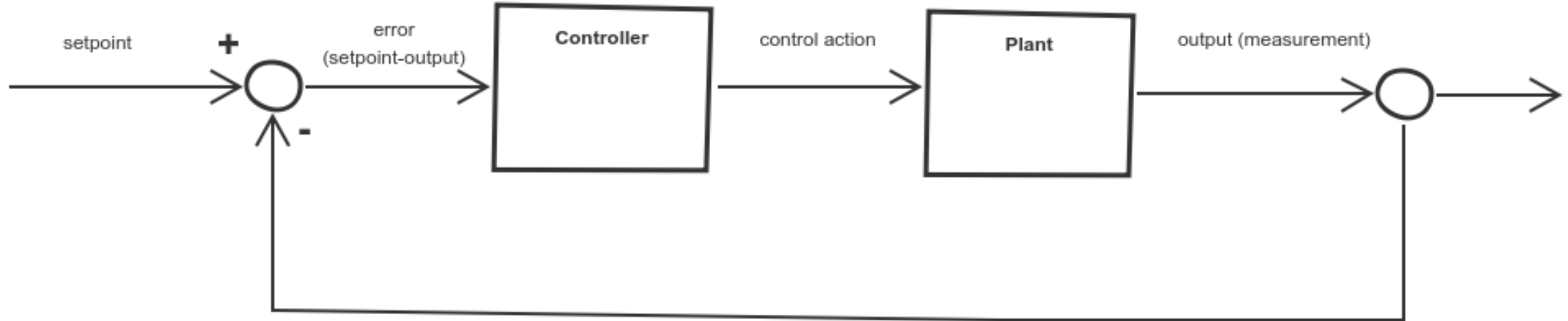


```
defmodule Component do
  @callback init(any) :: state
  @callback step(state, input) :: {output, state}
end
```

Feed-forward (open loop) control



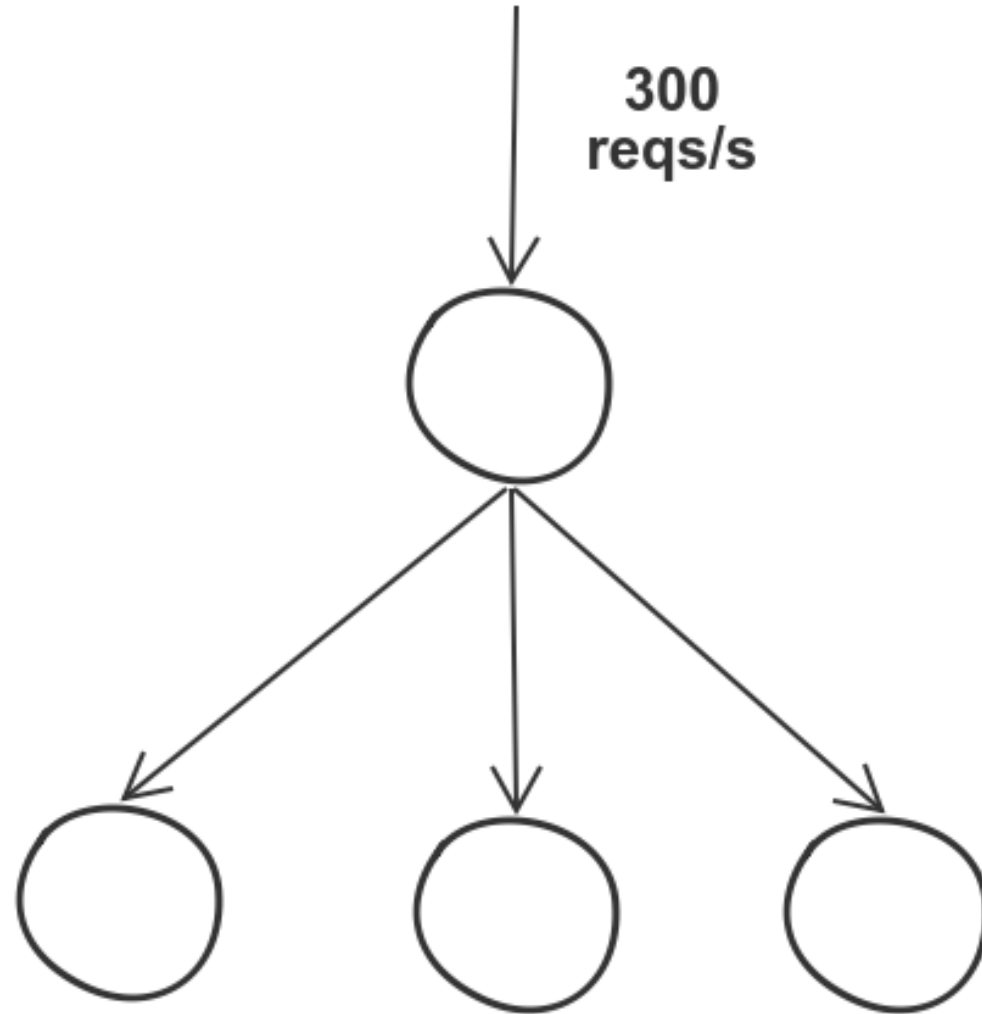
Feedback (closed loop) control



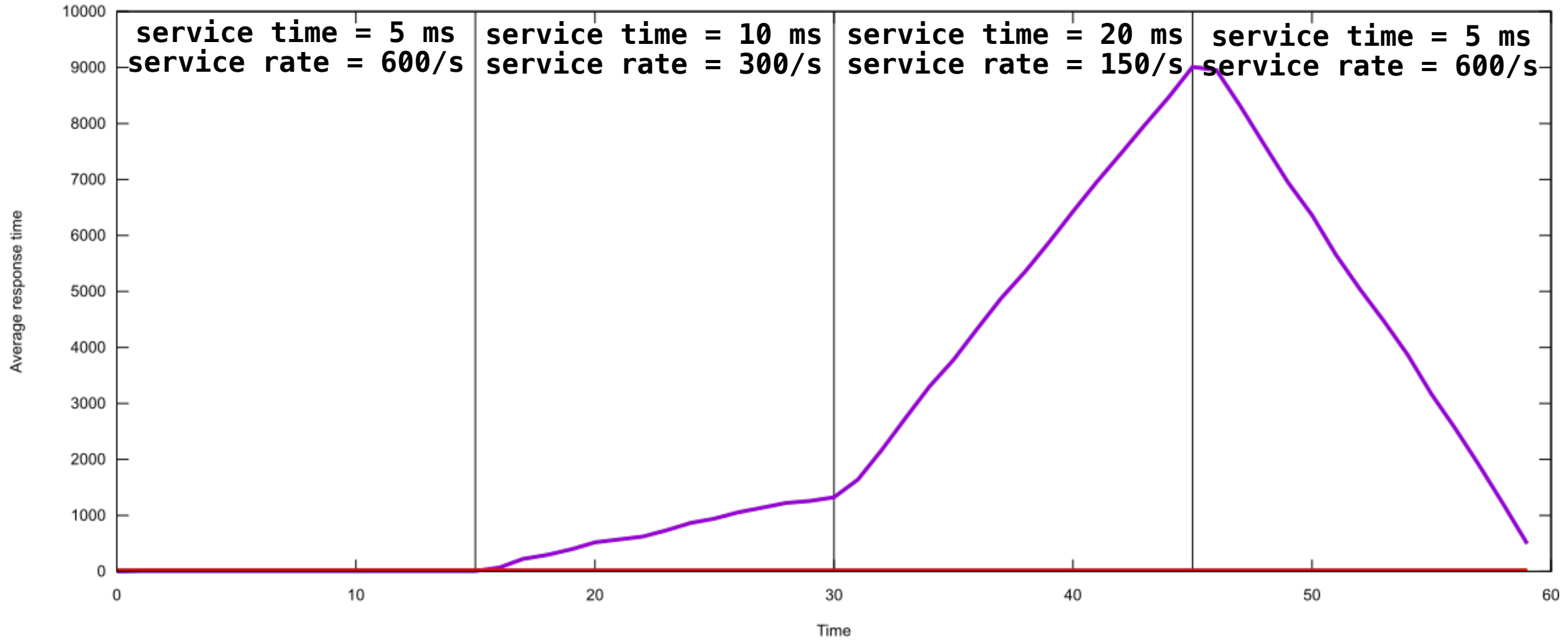
Feedback (closed loop) control

- Thermostat
- Cruise control
- Autopilot
- AWS Auto Scaling
- HTTP Live Streaming
- Redis key eviction
- TDD

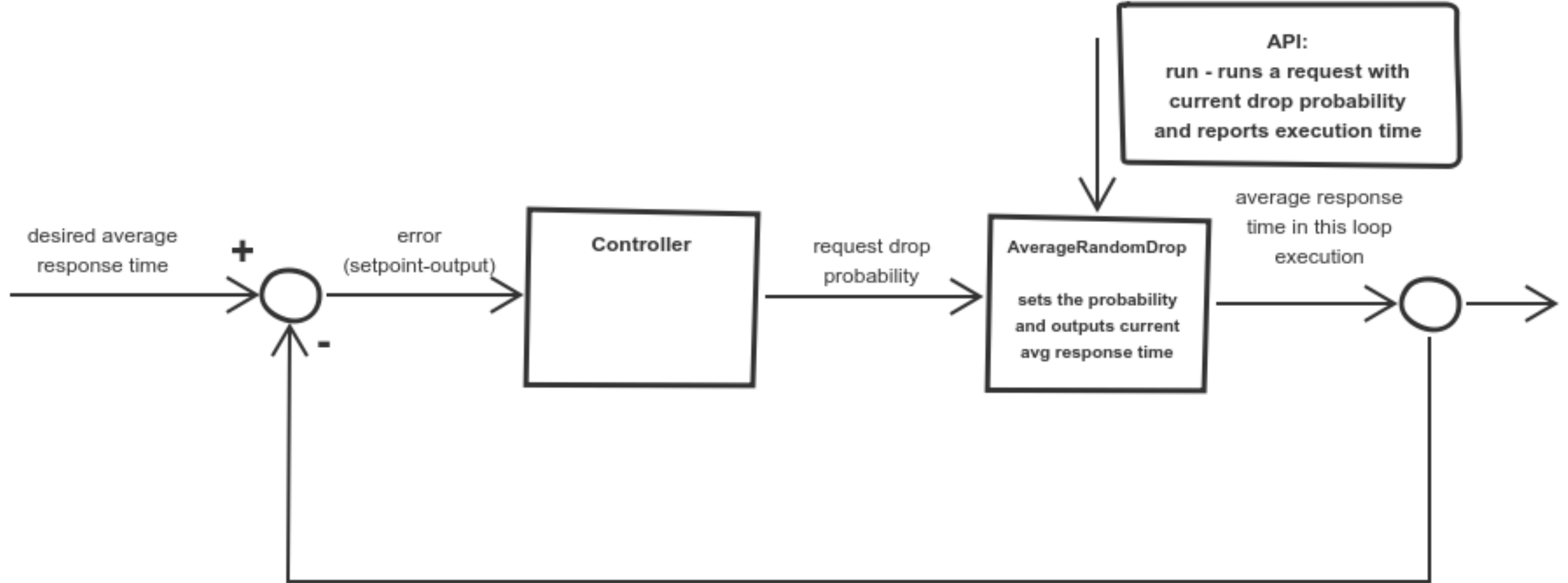
Three queues simulated



Three queues, naive approach



Random dropping based on average response time





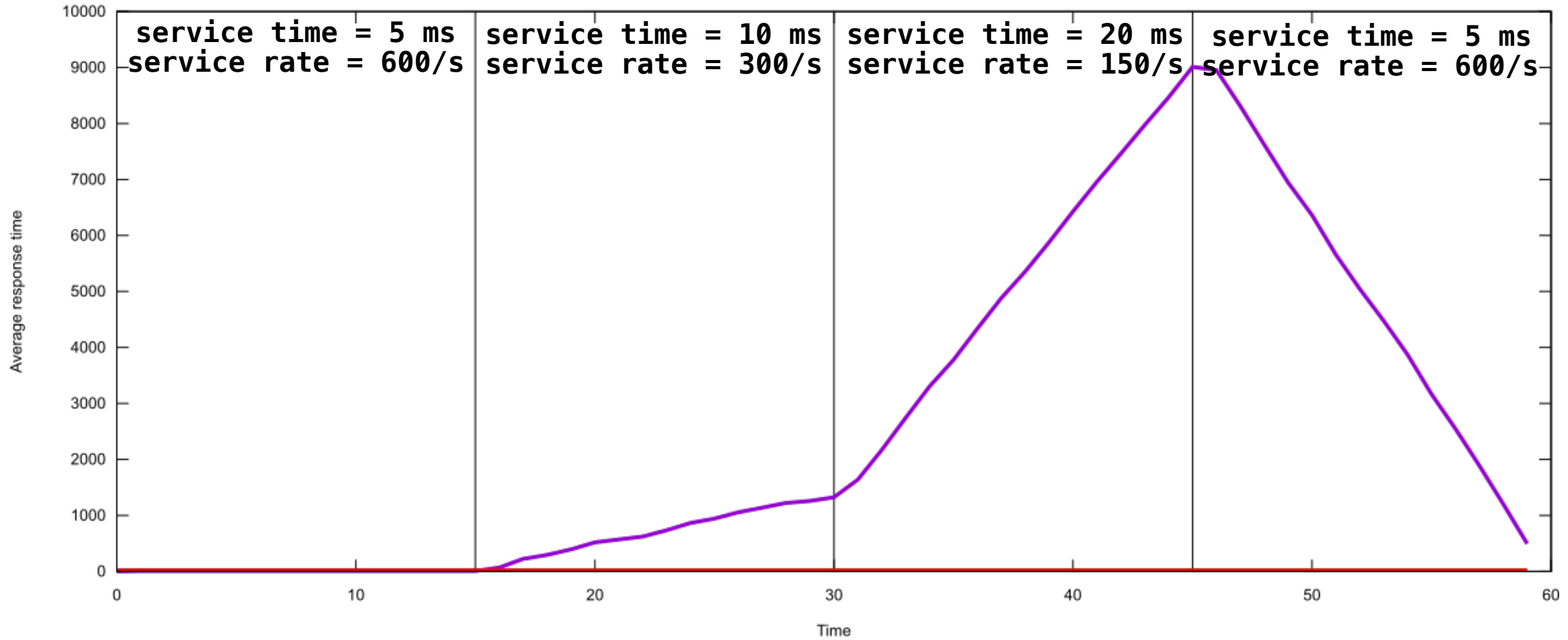
Proportional control

```
defmodule Controller do
  @behaviour Component

  def init(_), do: :no_state

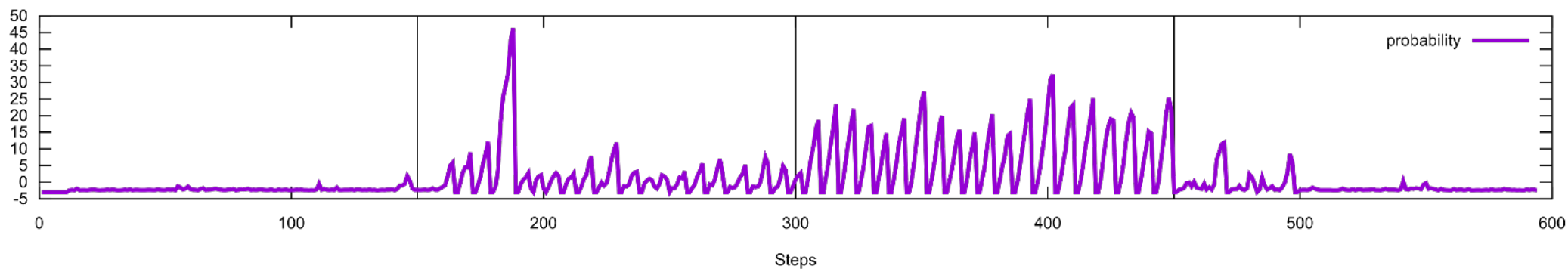
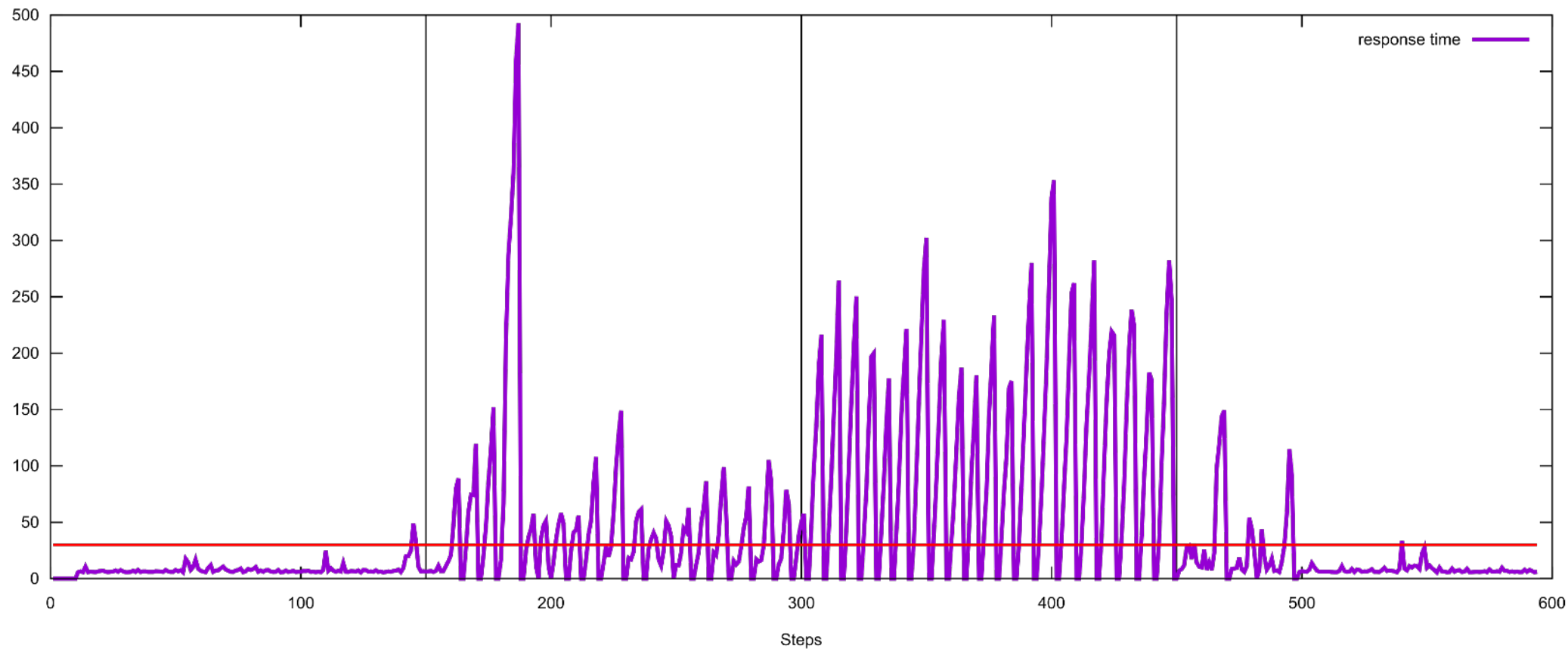
  def step(st, input) do
    {input*@magic_number, st}
  end
end
```


Three queues, naive approach



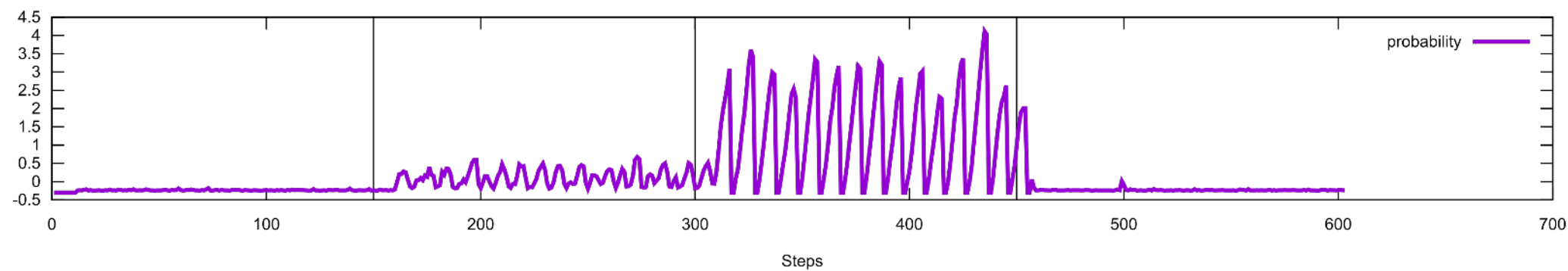
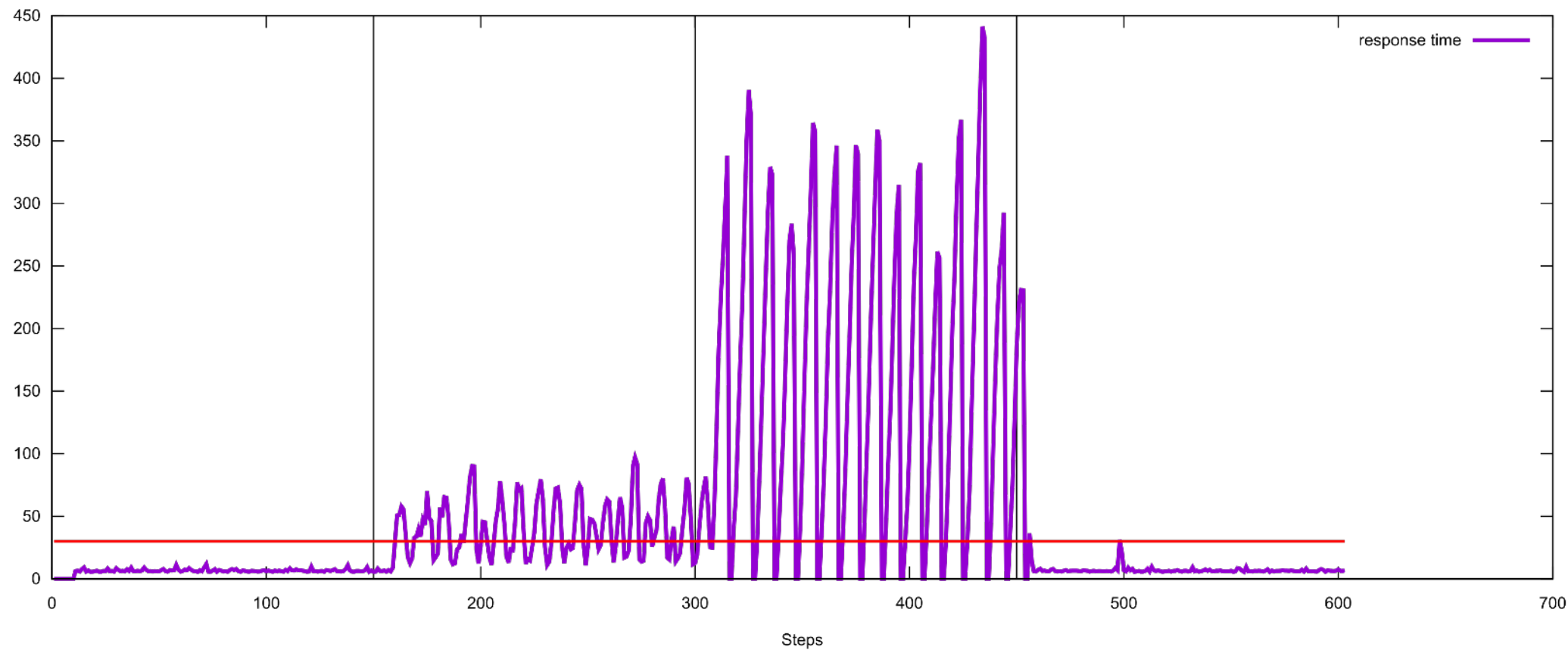


magic_number = 0.1

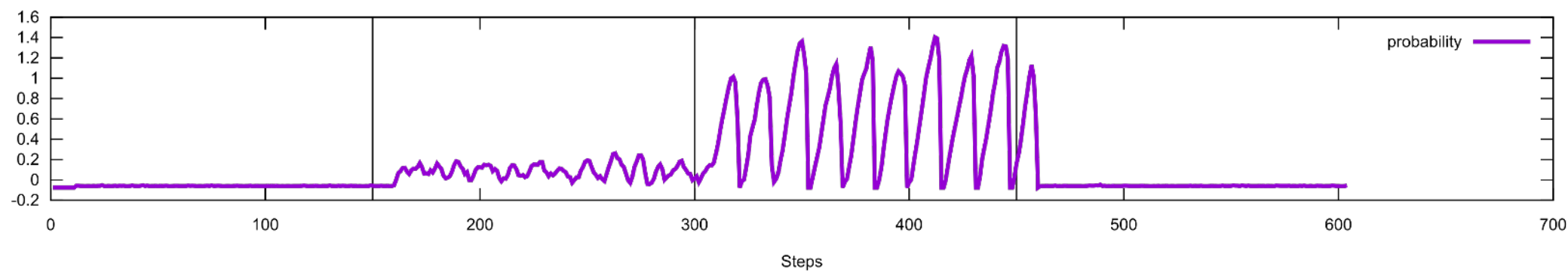
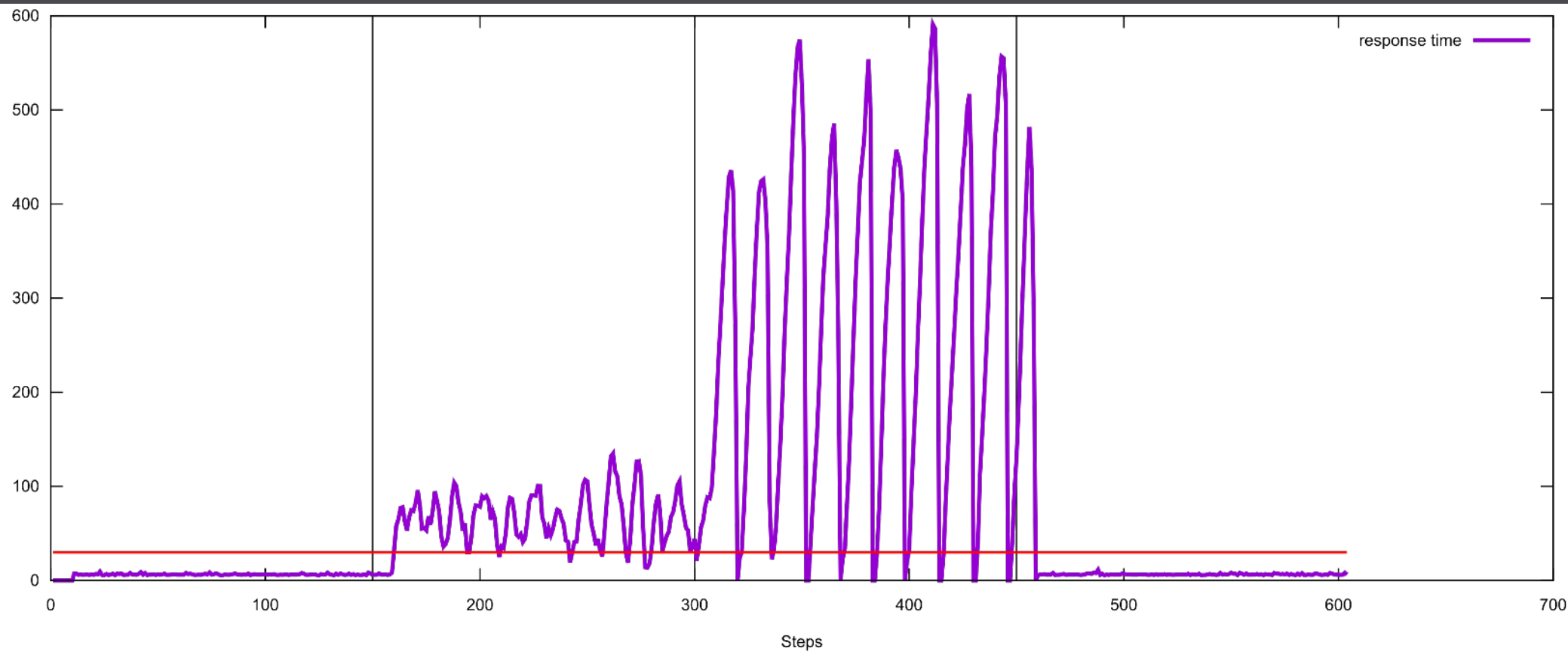




magic_number = 0.01



 **magic_number = 0.0025**



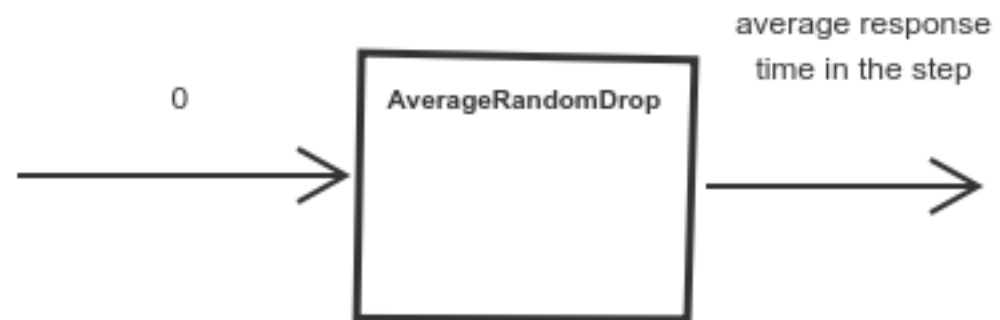
Proportional and integral controller

```
defmodule Controller do
  @behaviour Component

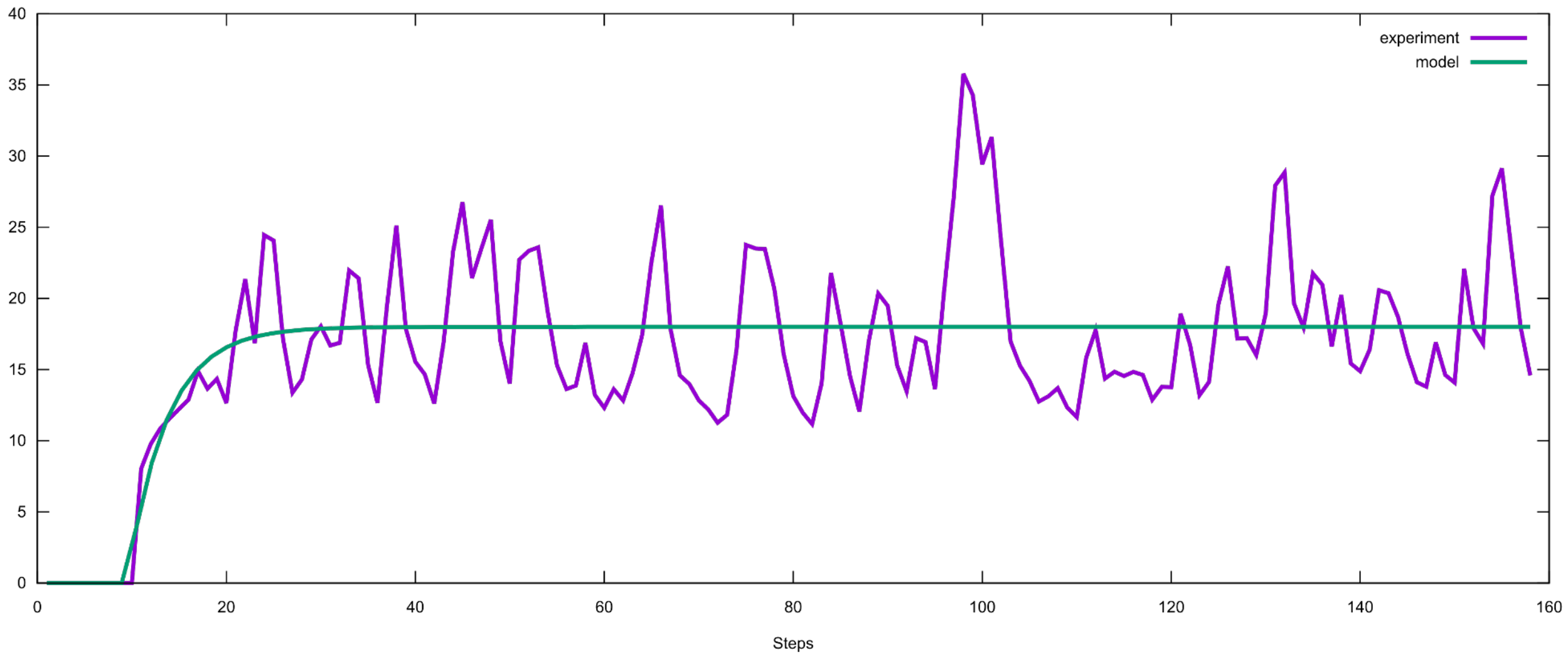
  def init(_), do: 0

  def step(i, input) do
    {input*@magic_number1 + (i+input)*@magic_number2, i+input}
  end
end
```

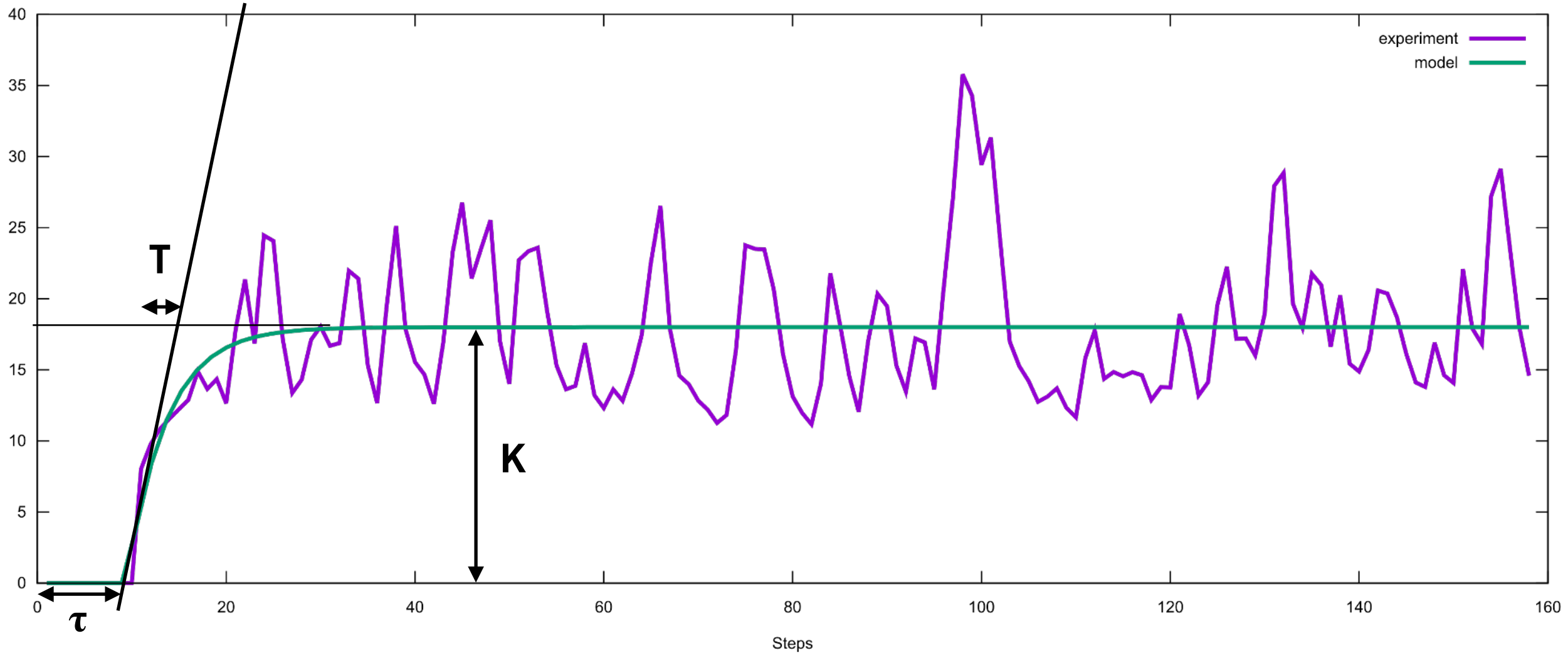
Step response experiment



Step response experiment



Step response experiment



Step response experiment

K = 18.0016

T = 4.16737

tau = 9.46876

Step response experiment

K = 18.0016

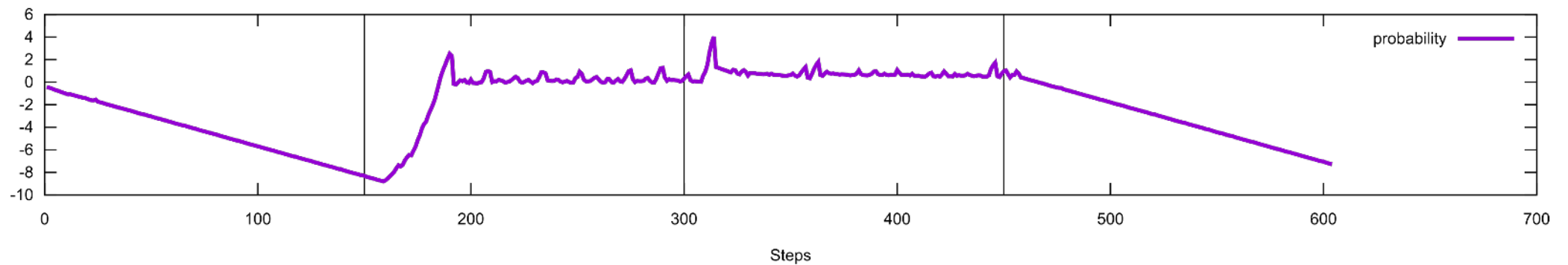
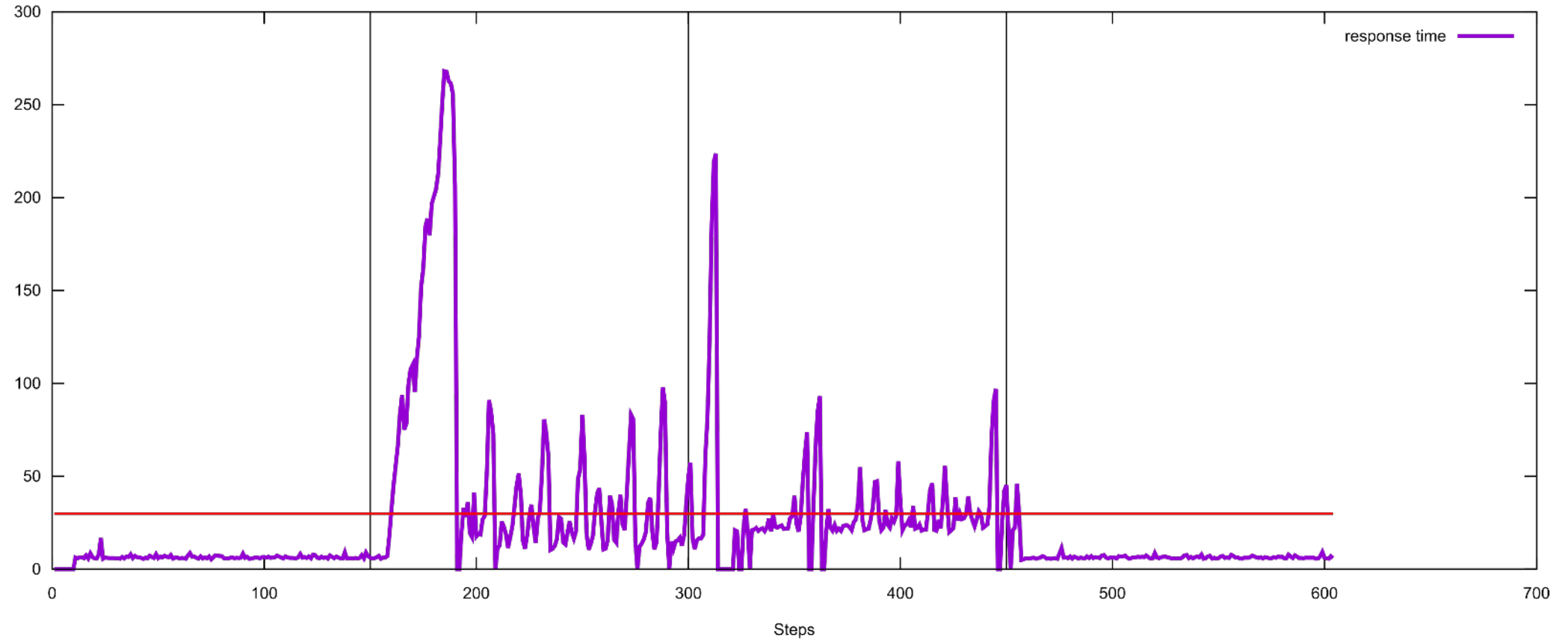
T = 4.16737

tau = 9.46876

magic_number1 = p = 0.002239326835923445

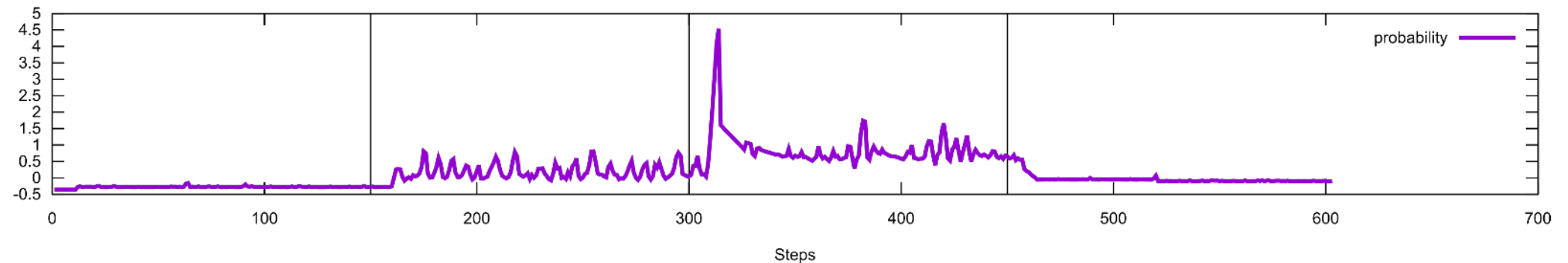
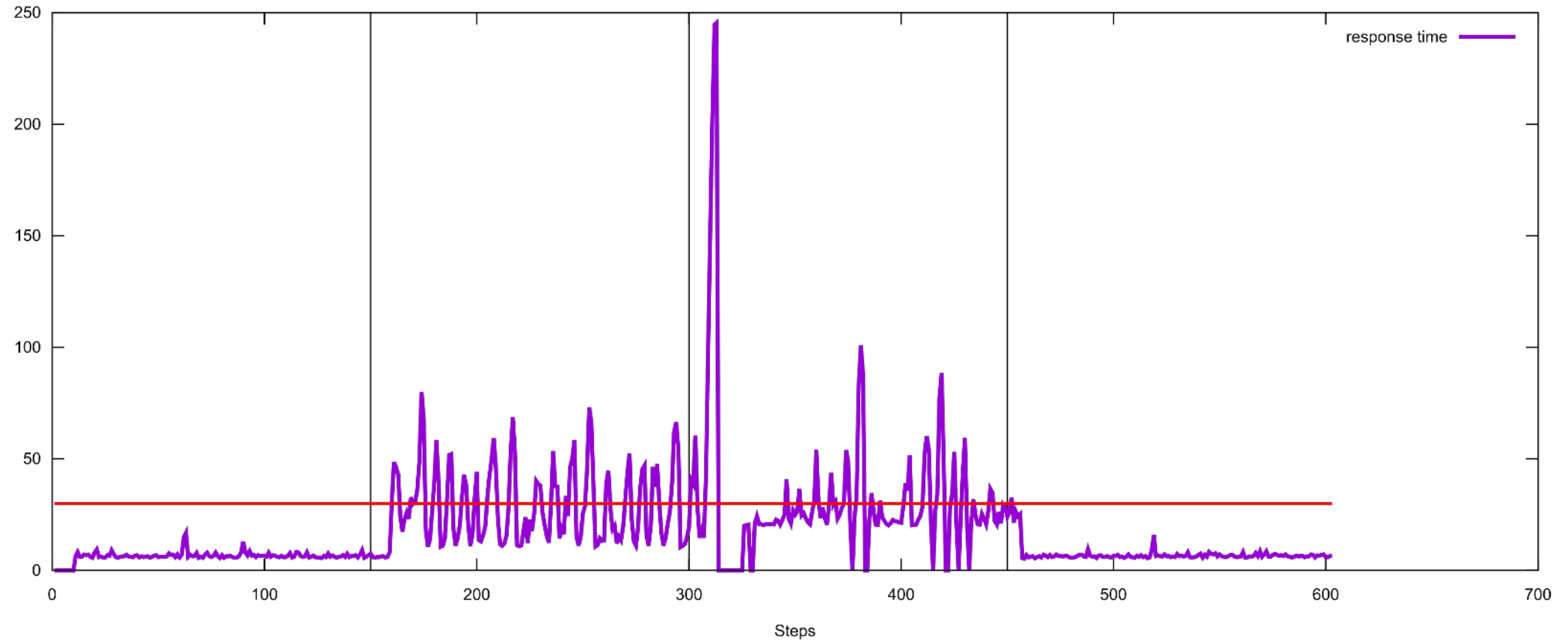
magic_number2 = i = 0.007701310787354867

Proportional and integral control



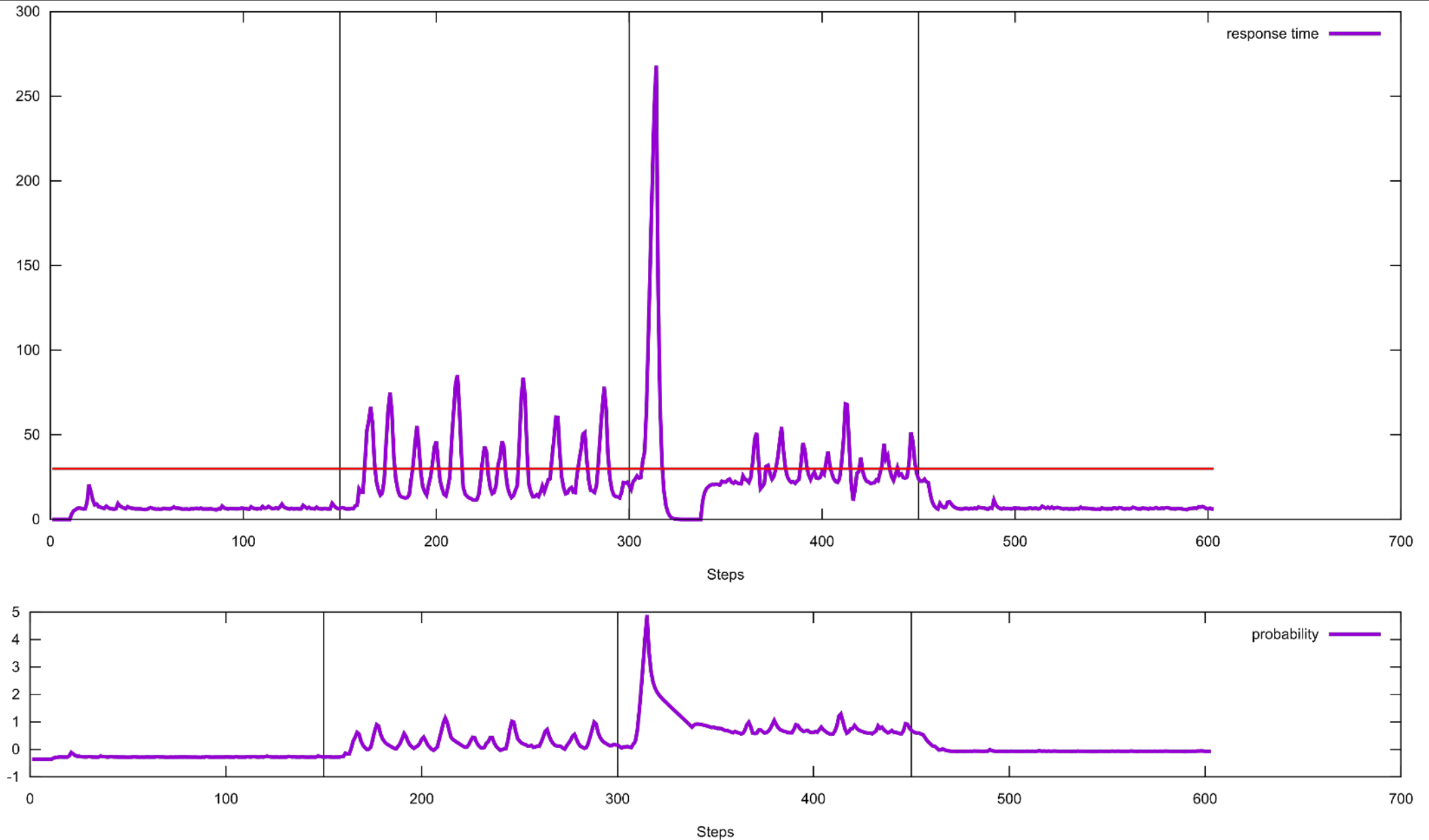


Proportional and integral control, with conditional integration



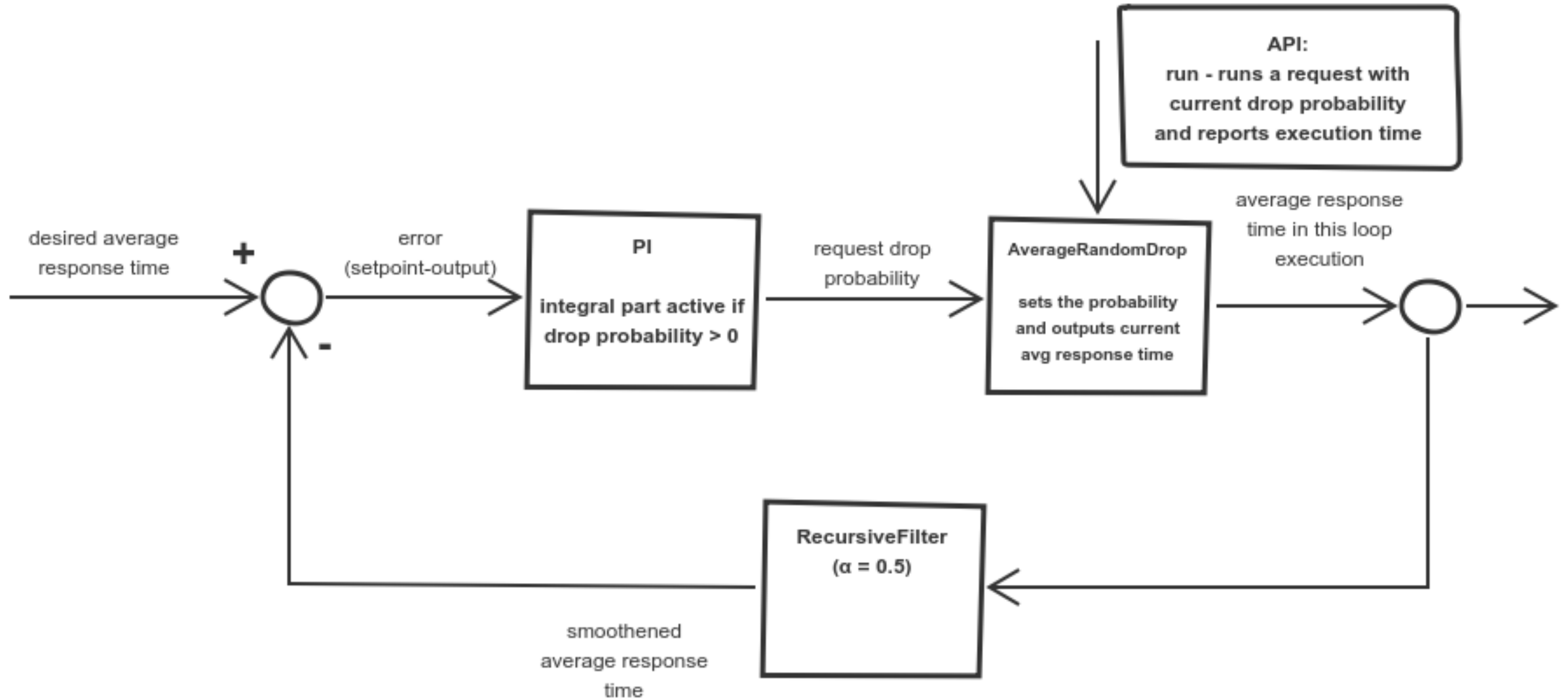


Proportional and integral control, with smoothing

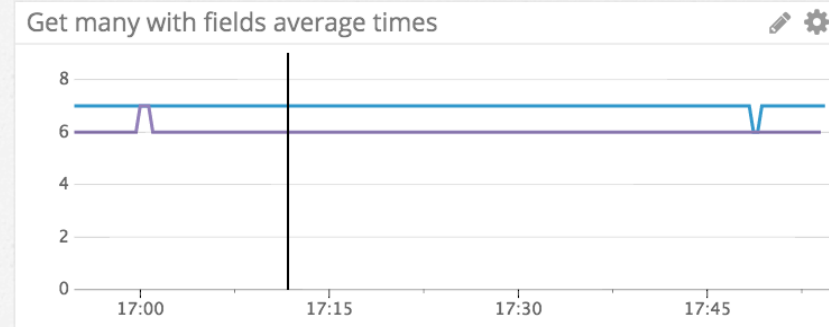
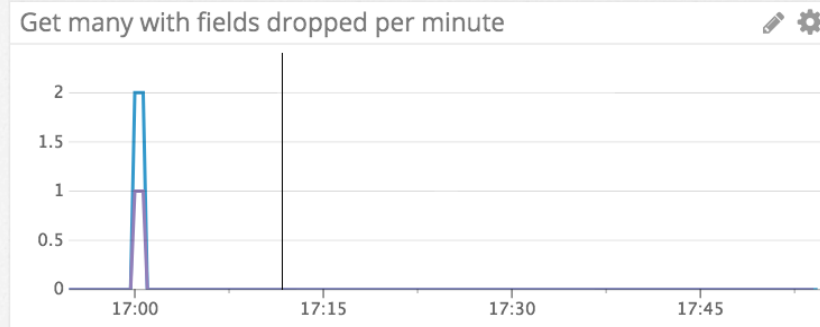
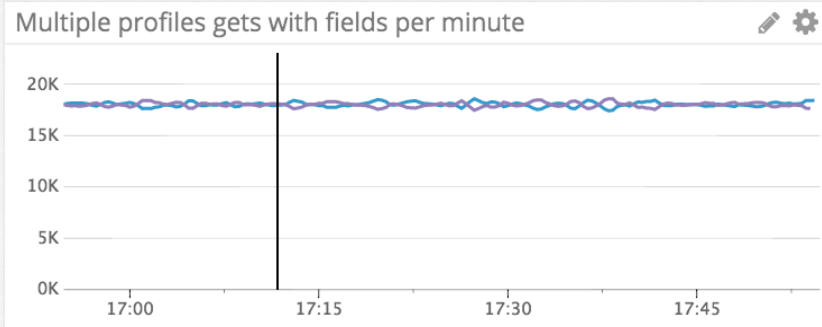




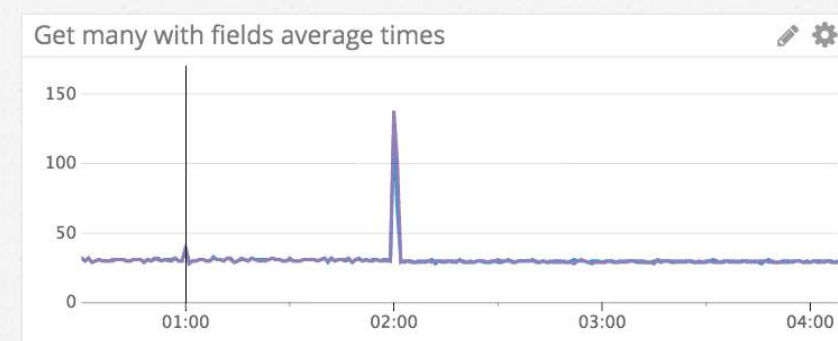
Final Loop



Simulation results #1 - steady state



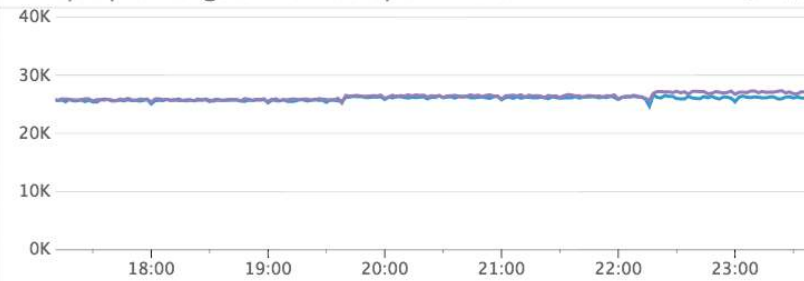
Simulation results #2 - load increase



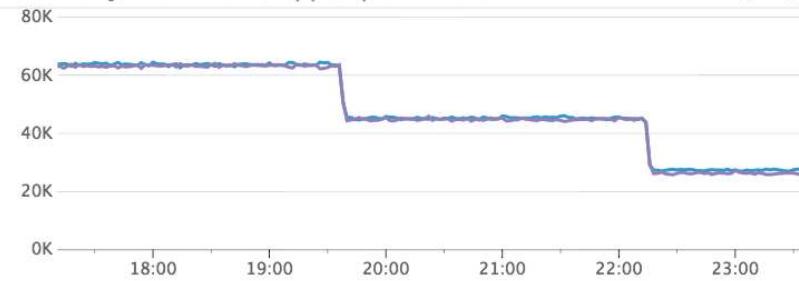


Simulation results #3 - load decrease

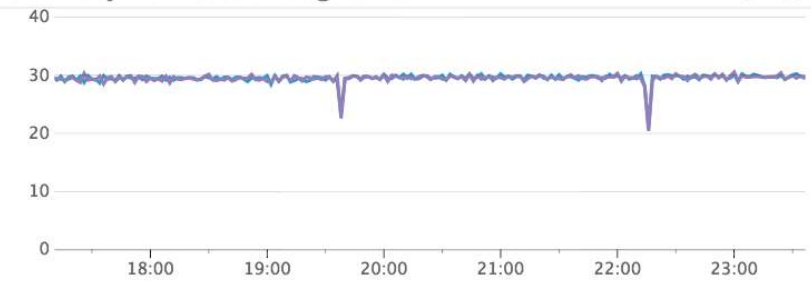
Multiple profiles gets with fields per minute



Get many with fields dropped per minute



Get many with fields average times



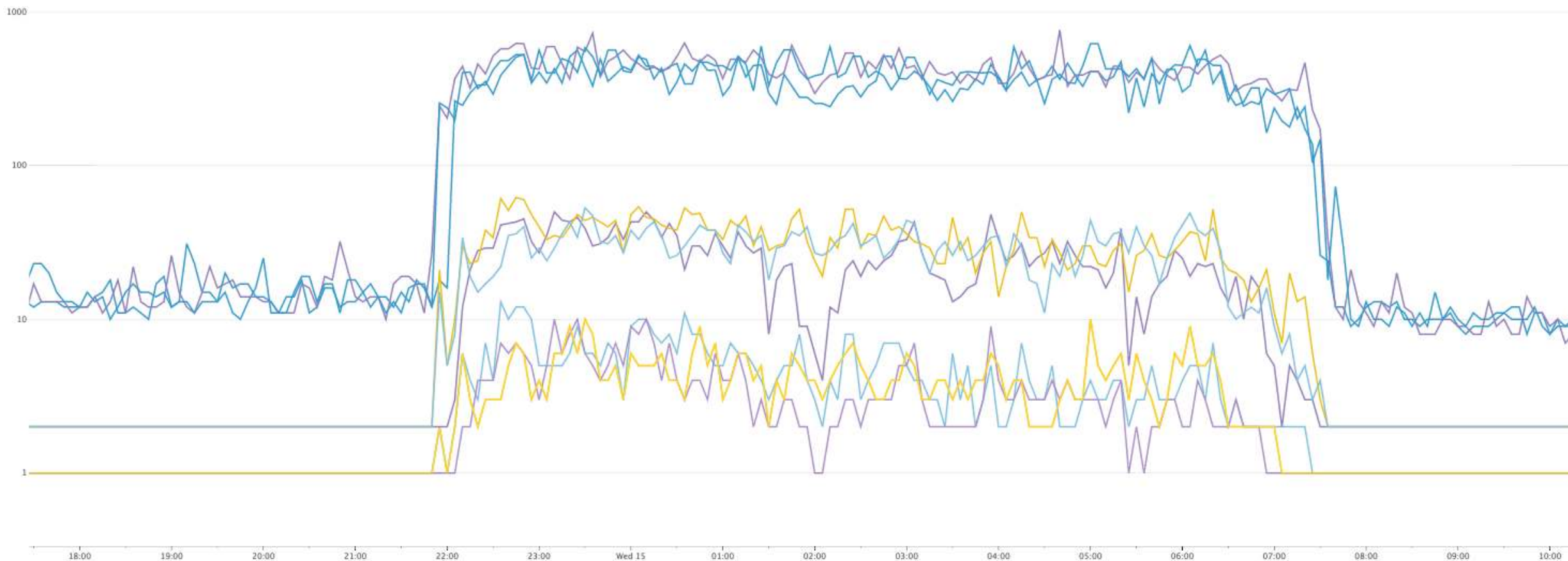
Real world examples



Real world example #1

Batch subscription time 100%/99%/95% (ms)

Show 17h Nov 14, 5:27PM - Nov 15, 10:17AM

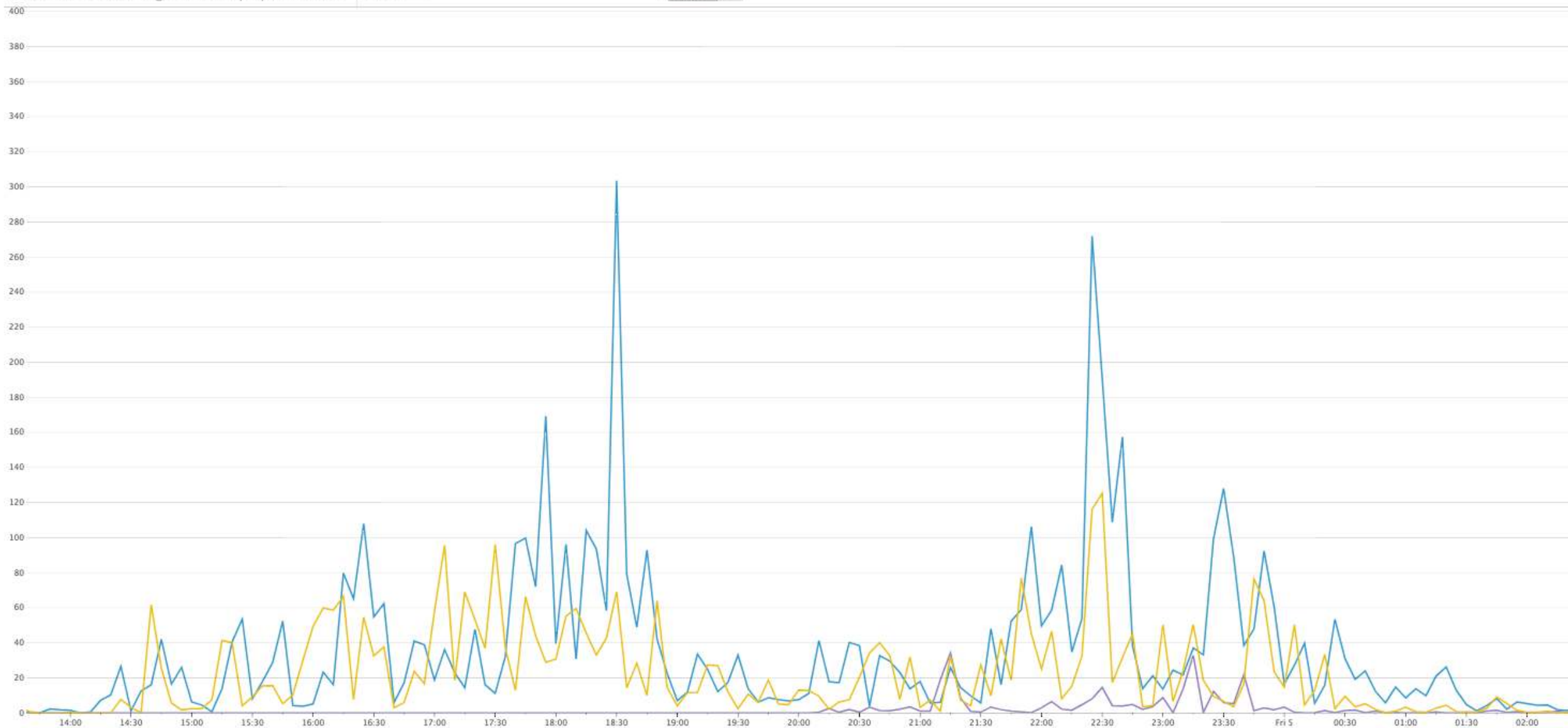




Real world example #1

Presence Tracker regulator drops per minute

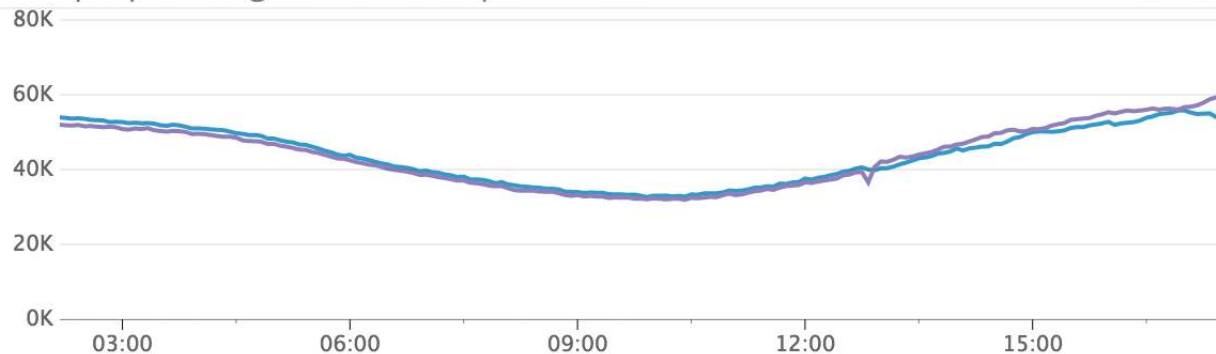
Show 1h The Past Hour



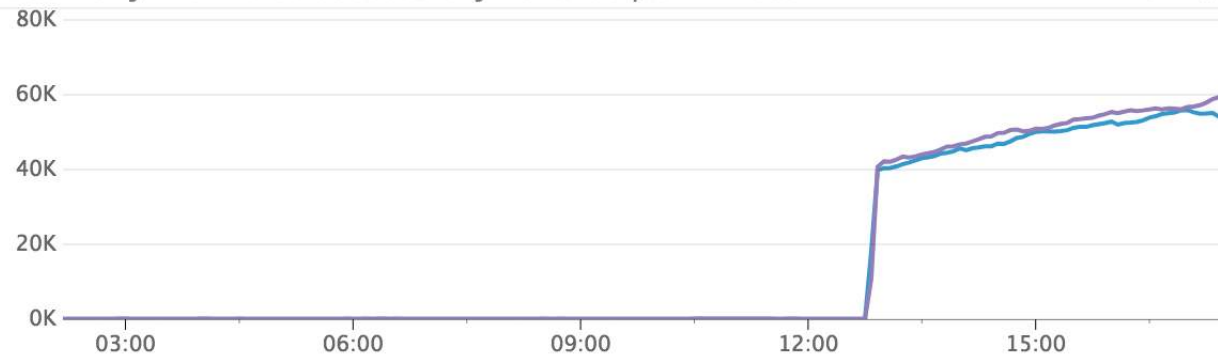


Real world example #2

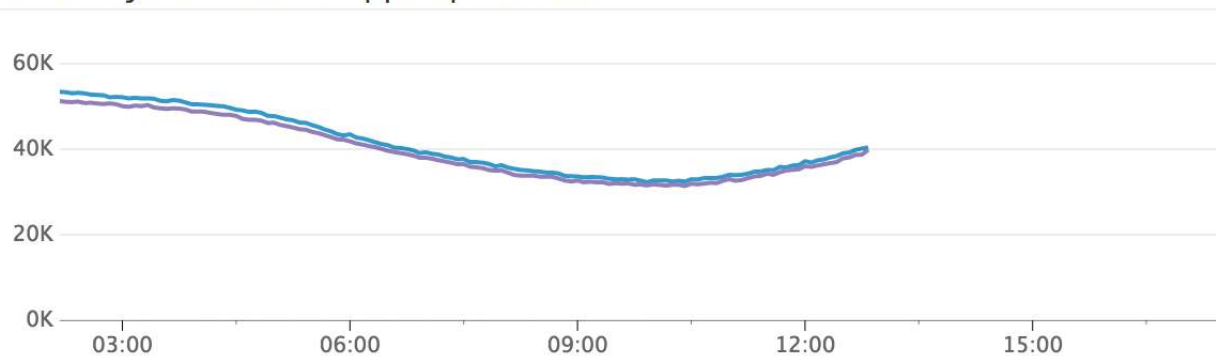
Multiple profiles gets with fields per minute



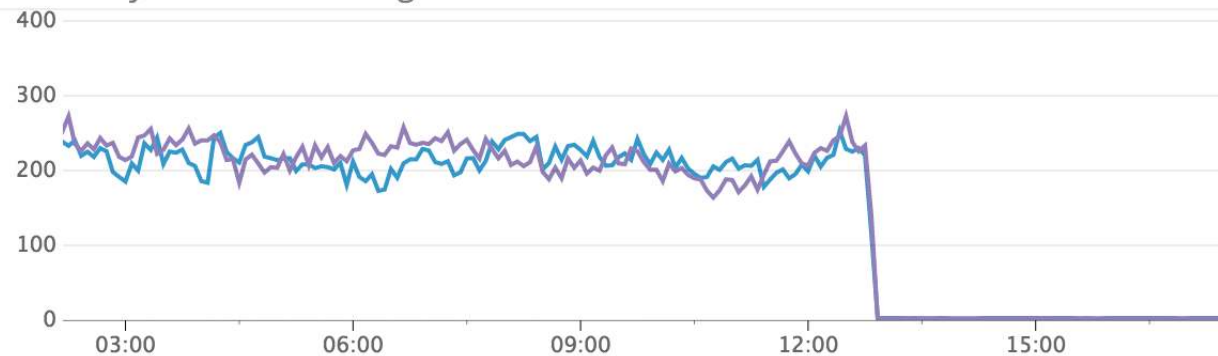
Get many with fields successfully handled per minute



Get many with fields dropped per minute



Get many with fields average times



What else?

- Use this approach in all the backend services, in all points of uncertainty.
- Use it in the client app, for better UX, by providing more lightweight content for low-bandwidth networks.

References

- *Feedback Control for Computer Systems* by Phillip K. Janert
- [Control System Lectures](#) by Brian Douglas
- [Queues don't fix overload](#) by Fred Hebert
- [Handling overload](#) by Fred Hebert
- <https://github.com/fishcakez/sbroker>
- <https://github.com/ferd/dispcount>
- <https://github.com/uwiger/jobs>
- [Surfing on Lava](#)

 **Questions?**
